PA2 — Matrix Transformations and Perspective

Student ID: 241482

Name : Amina Smailova

E1 — Order of Transformations

Original matrix product:

M = R_orbit · T · R_self · S

Experiment A — Swap orbit rotation and translation

Prediction:

If the orbit rotation and translation are swapped, I expect the Wedge to stop orbiting around the cube normally. It should stay at a fixed translated position while its orientation changes.

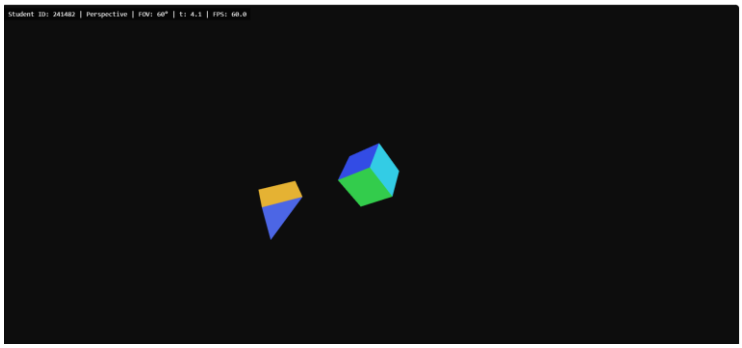


Figure E1(a): After swapping translation and orbit rotation, the Wedge stays at a fixed translated position instead of orbiting the cube.

Result:
After swapping the orbit rotation and translation, the Wedge no longer orbited around the cube. It stayed at approximately the same translated position while its orientation continued to change.

Explanation:
The new matrix order is M = T · R_orbit · R_self · S. glMatrix right-multiplies each new transformation, so the last transformation in the product is applied to the vertex first. Because the translation is now outside the orbit rotation, the translation vector is not rotated around the origin. So the centre of the Wedge stays at a fixed translated position instead of following a circular orbit.

Experiment B — Swap translation and self-spin

Prediction: If the translation and self-spin are swapped, I expect the Wedge to stop following the intended vertical circular orbit. The self-spin rotation will also rotate the translation vector, so the position of the Wedge will be affected by both rotations.

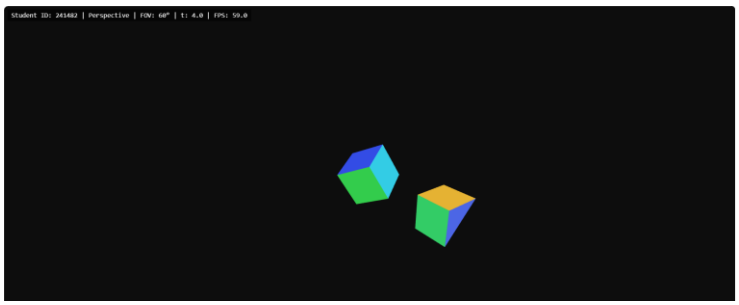


Figure E1(b): After swapping translation and self-spin, the Wedge follows a different combined trajectory because the self-spin also rotates its translated position.

Result:
After swapping the translation and self-spin, the Wedge no longer followed the original simple vertical orbit. Its trajectory changed because the self-spin rotation also affected the translation vector.

Explanation:
The new matrix product is M = R_orbit · R_self · T · S. Since glMatrix right-multiplies transformations, the translation is applied to the vertex before the self-spin rotation. So R_self rotates not only the Wedge itself but also its translated position. As a result, the centre of the Wedge follows a different combined trajectory instead of the original vertical circular orbit.

E2 — Three Statements in Chapter 4

(a) Rotation axis correction

The comment says that the rotation is around the X-axis, but the code uses [0, 1, 0].

This vector means the Y-axis, not the X-axis. So the correct comment should say that the rotation is around the Y-axis.

(b) Clipping vs face culling

Clipping and face culling are different things.

Clipping happens when part of an object is outside the camera view. The part outside the viewing area is removed, but the visible part can still be shown.

Face culling removes a whole face of an object. For example, back-face culling can remove faces that are turned away from the camera.

So, objects outside the viewing frustum are handled by clipping, not face culling.

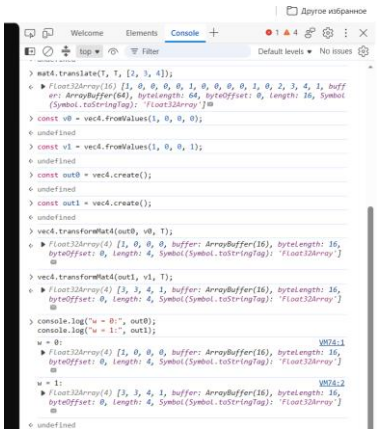


Figure E2(c): Console output for vectors with w = 0 and w = 1 after applying the translation matrix.

I used a translation matrix with the translation [2, 3, 4].

For the vector [1, 0, 0, 0], the result was [1, 0, 0, 0].

For the vector [1, 0, 0, 1], the result was [3, 3, 4, 1].

When w = 0, the vector acts like a direction, so translation does not change it.

When w = 1, the vector acts like a point, so the translation changes its position.

So, w shows whether translation should affect the vector.

E3 — Perspective vs Orthographic

Prediction:

In perspective projection, the nearer edge of the cube should look larger than the farther edge. In orthographic projection, both edges should be almost the same size.

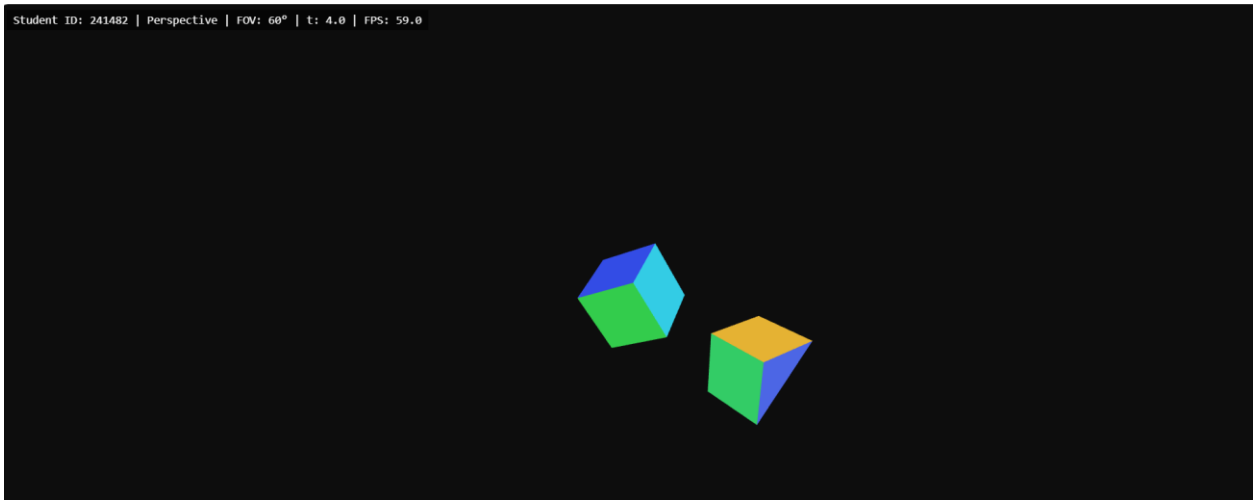


Figure E3(a): Perspective projection at t = 4.0.

Result:

In perspective projection, the nearest vertical edge was about 85 px and the farthest vertical edge was about 81 px.

Ratio: 85 / 81 ≈ 1.05.

Explanation:

In perspective projection, objects that are closer to the camera look larger. This happens because of the perspective divide, where the coordinates are divided by w.

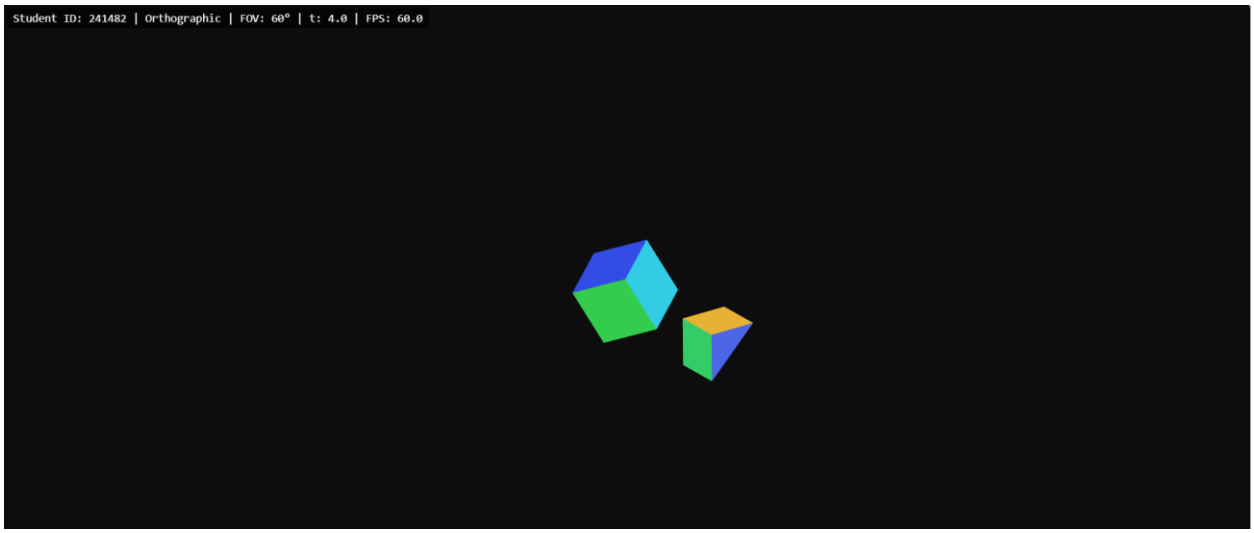


Figure E3(b): Orthographic projection at the same pose, t = 4.0.

In orthographic projection, both vertical edges were about 81 px.

Ratio: 81 / 81 ≈ 1.00.

Explanation:

In orthographic projection, the size does not change with distance, so the near and far edges have almost the same length.

The PA1 depth offset trick is more similar to orthographic projection because the back part was only shifted, but it didn’t become smaller with distance

E4 — Near, Far and Aspect

In my normal scene I used:

near = 0.1
far = 30.0

During one full orbit, the closest vertex is about 5.46 units from the camera and the farthest vertex is about 10.25 units away. I chose 0.1 for near because it is much closer than any object in the scene. I chose 30 for far because it is much farther than the farthest object. So the whole scene stays visible during the animation.

(a) Near plane experiment

Prediction:

If I increase the near value so that the near plane passes through the scene, I expect parts of the objects to disappear or look cut.

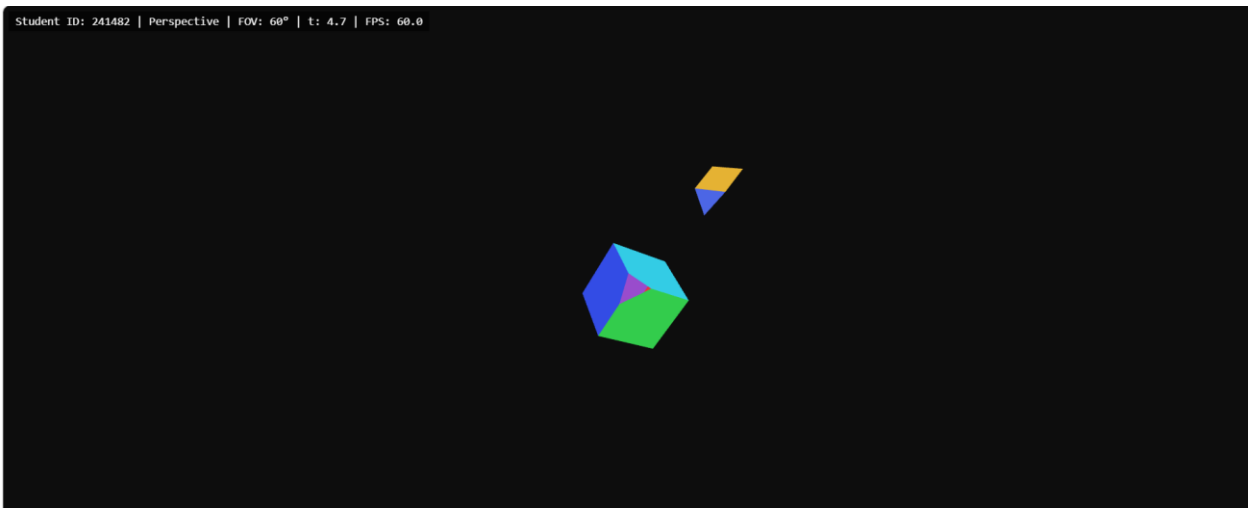


Figure E4(a): With near = 7.0, parts of the objects are clipped when they come too close to the camera.

Result:

After changing near from 0.1 to 7.0, parts of the objects were cut when they came too close to the camera.

This happens because geometry in front of the near clipping plane is not rendered.

(b) Aspect ratio experiment

Prediction:

If I set aspect = 1 and make the browser window wide, I expect the scene to look stretched because the projection will not match the real window shape.

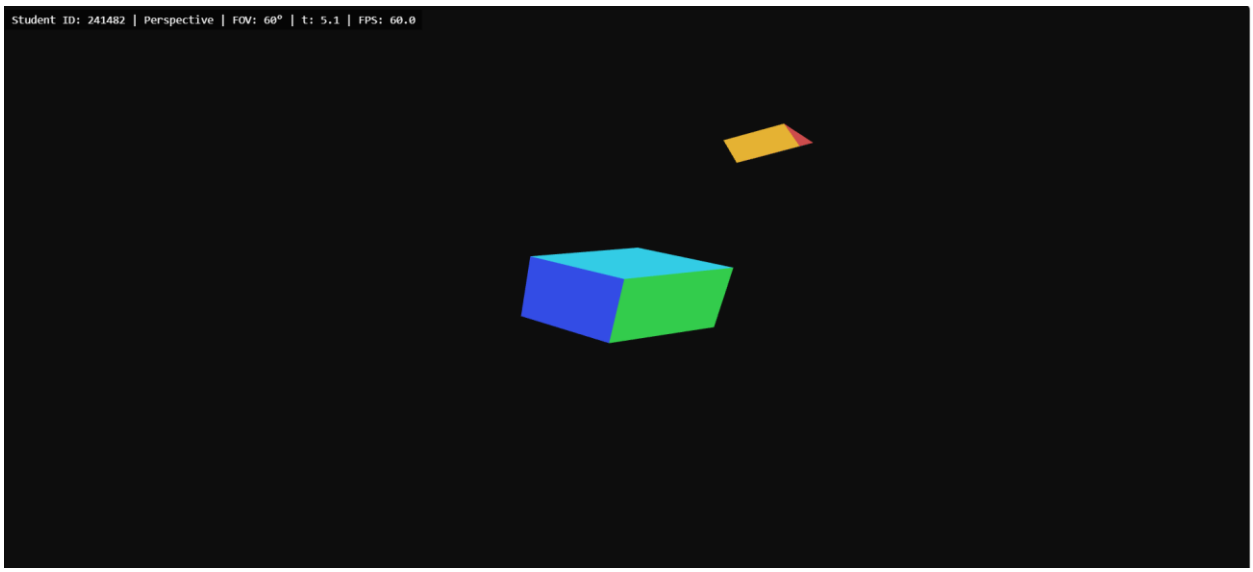


Figure E4(b): The scene is distorted after hard-coding aspect = 1 in a wide browser window.

Result:

After setting aspect = 1 and making the browser window wide, the cube looked stretched.

This happened because the projection used a square aspect ratio, but the real browser window was much wider.

The correct aspect ratio should be calculated as width divided by height.

E5 — Frame-rate Independence

a) Prediction:

The animation should use real time, not the number of frames.

If the frame rate becomes lower, the animation may look less smooth, but one full orbit should still take about 8 seconds.

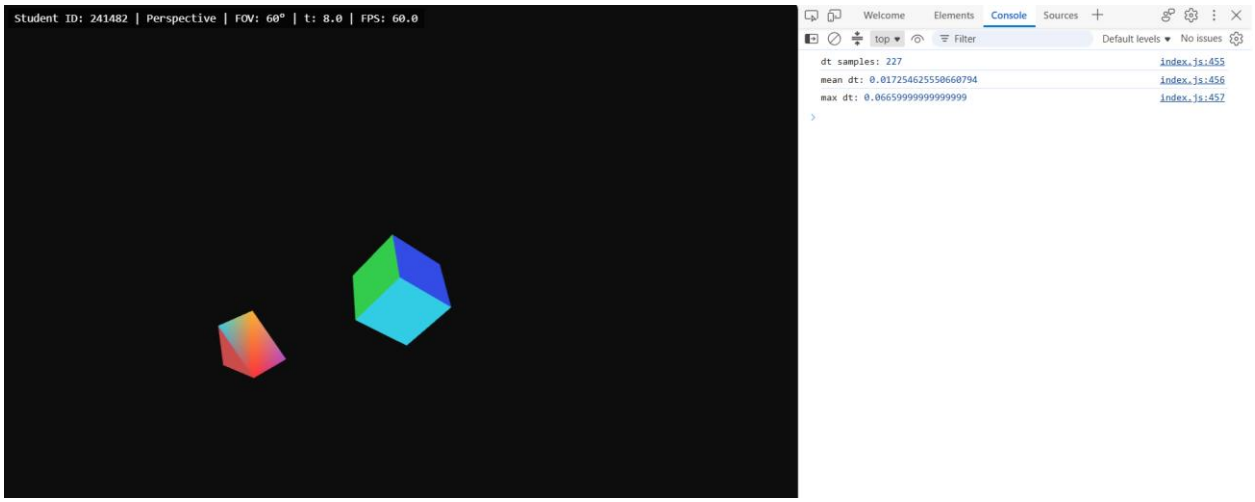


Figure E5(a): Console output showing the mean and maximum dt values.

(a) Delta time

I logged dt values for about 5 seconds.

Number of samples: 227
Mean dt: about 0.0173 s
Maximum dt: about 0.0666 s

The mean dt is close to the time between frames at around 60 FPS. The maximum value was higher because some frames took longer than others.

(b) CPU throttling

Prediction:

With CPU throttling, the animation may look less smooth and FPS may become lower, but one full orbit should still take about 8 seconds because the animation uses dt.

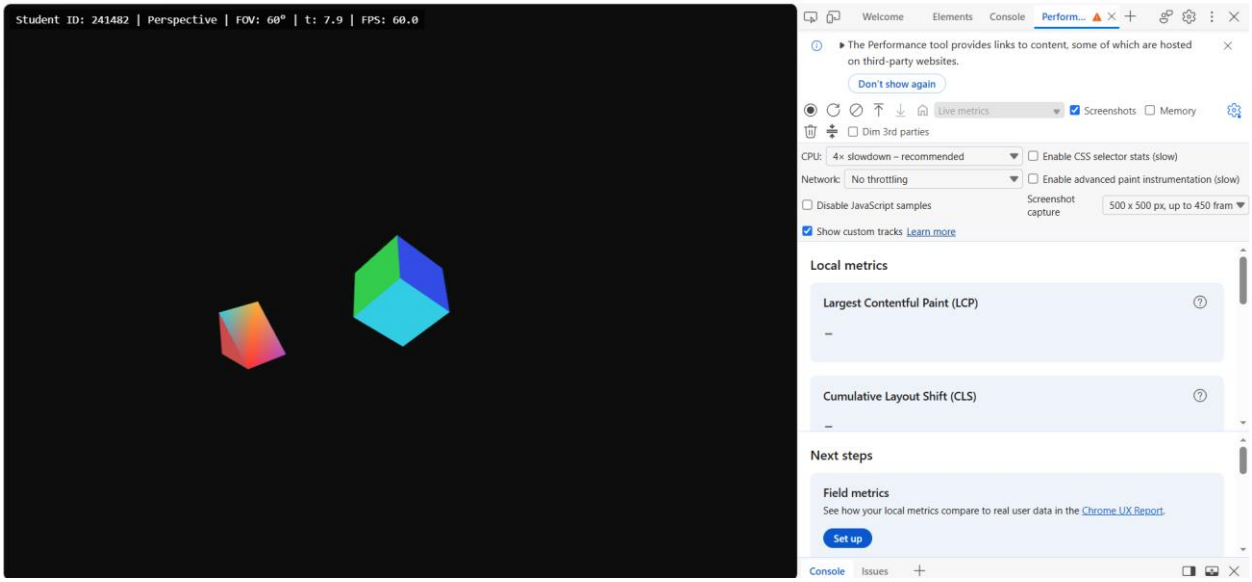


Figure E5(b): With 4x CPU slowdown, the Wedge still completes one orbit in about 8 seconds.

I enabled 4x CPU slowdown in DevTools.

The animation became less smooth, but the Wedge still completed one full orbit in about 8 seconds.

This shows that the animation depends on delta time, not on the number of frames.

(c) Removing the dt clamp

Prediction:

If I remove the dt clamp and switch to another tab for about 5 seconds, I expect the animation to jump forward when I come back because dt will become very large.

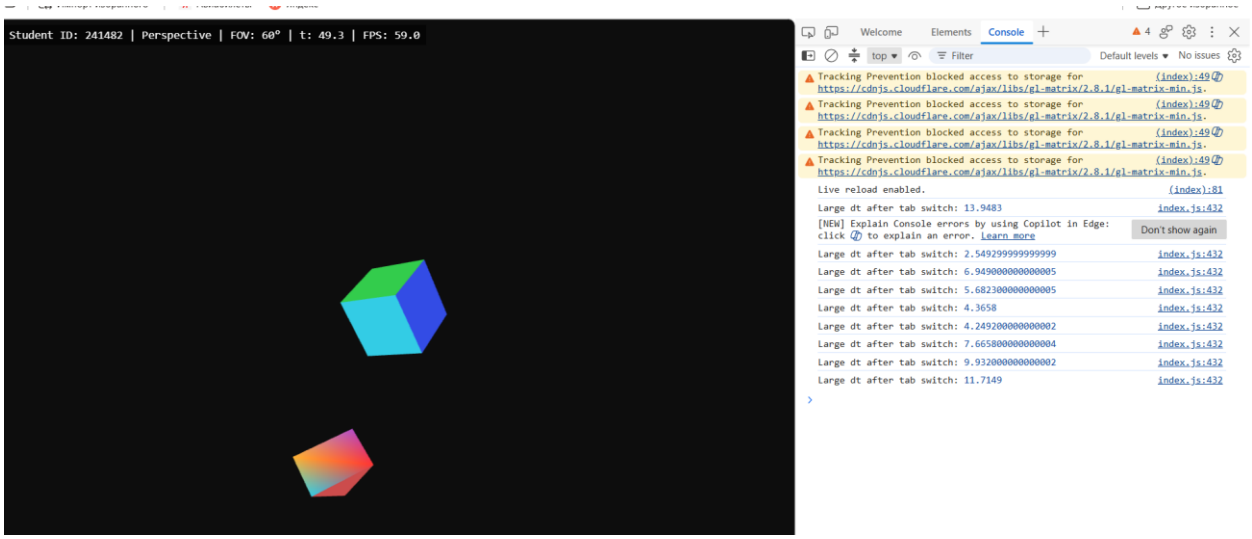


Figure E5(c): Without the dt clamp, switching tabs caused a large dt and the animation jumped forward.

Result:

After removing the dt clamp and switching to another tab, the dt became very large.

When I came back, the animation jumped forward because all the time spent on the other tab was added at once.

The dt clamp prevents this by limiting dt to a maximum of 0.1 seconds.

E6 — One Vertex by Hand

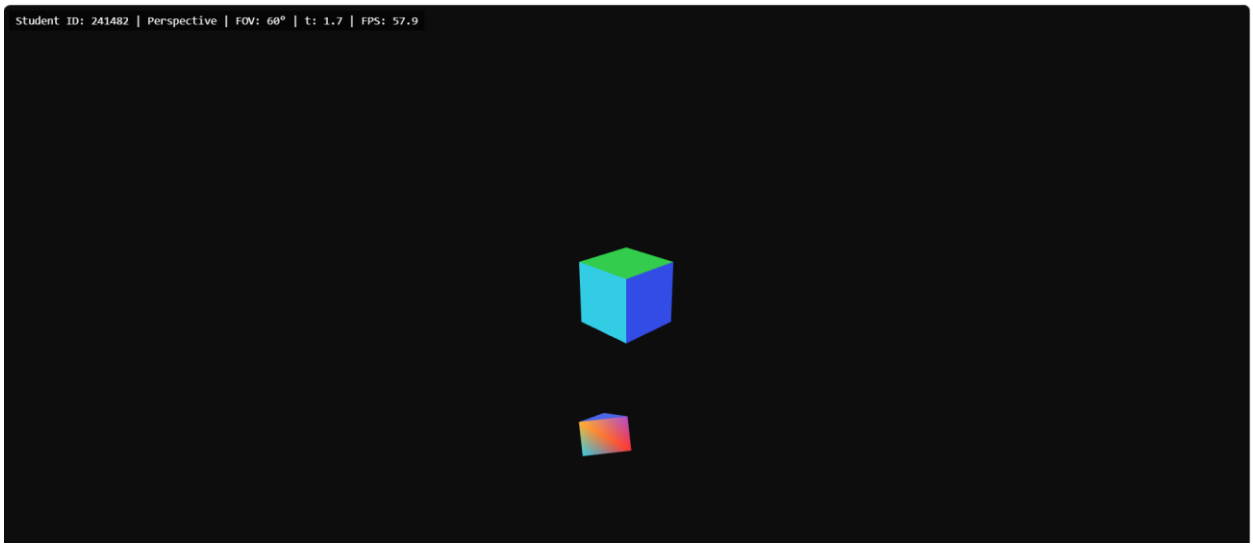
For my variant, the cube rotates around the normalized (1, 1, 1) axis.

I used a 120° rotation.

$t = (2\pi / 3) / 1.2 \approx 1.745$ s

The vertex is:

v = (0.5, 0.5, -0.5, 1)



For a 120° rotation around the (1, 1, 1) axis, the coordinates change as:

(x, y, z) → (z, x, y)

So:

(0.5, 0.5, -0.5) → (-0.5, 0.5, 0.5)

Therefore:

M · v = (-0.5, 0.5, 0.5, 1)

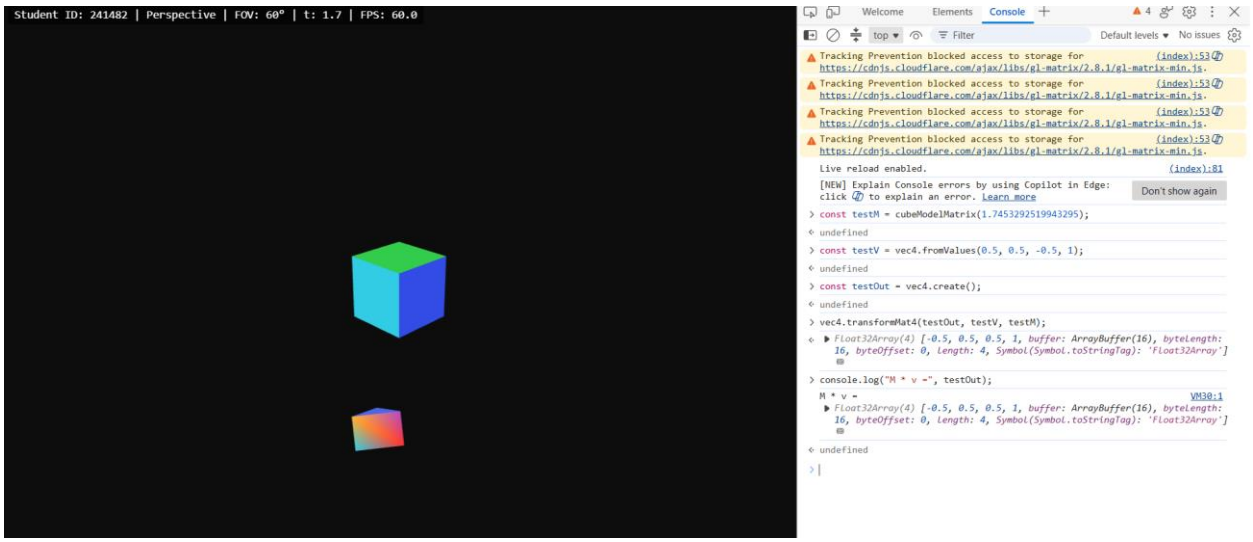


Figure E6(a): Console check of M · v for the vertex (0.5, 0.5, -0.5, 1) at a 120° cube rotation.

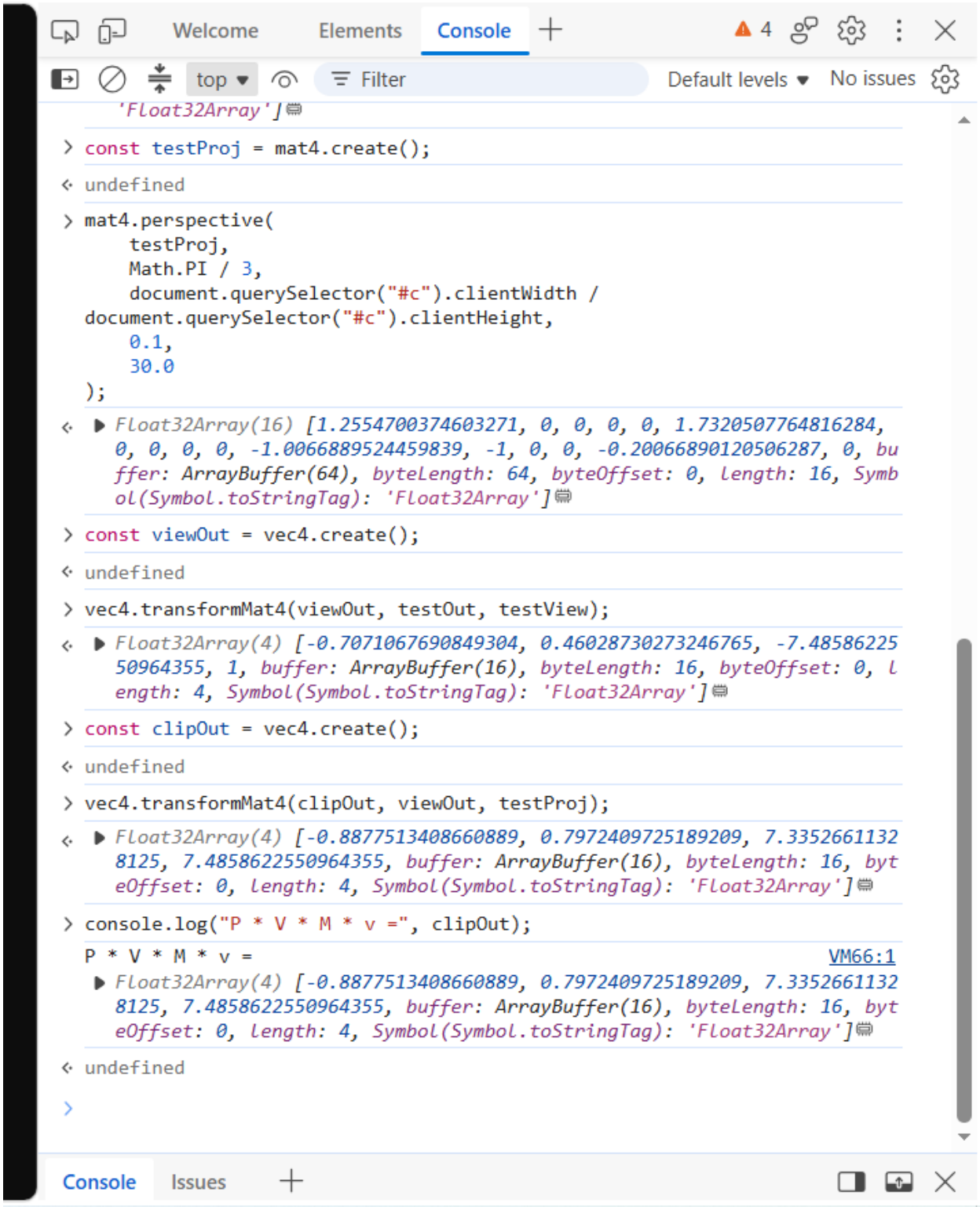


Figure E6(b): Console output for the full clip-space result P · V · M · v.

The full clip-space result was:

P · V · M · v = (-0.8878, 0.7972, 7.3353, 7.4859)

To get NDC, I divided x, y and z by w:

x = -0.8878 / 7.4859 ≈ -0.1186

$y = 0.7972 / 7.4859 \approx 0.1065$

$z = 7.3353 / 7.4859 \approx 0.9799$

So the NDC coordinates are approximately:

(-0.1186, 0.1065, 0.9799)

Development Log

During PA2, I had a problem with the old PA1 geometry because the cube was not centred at the origin.

I found the problem by checking the cube coordinates and comparing them with the PA2 requirements.

I changed the cube coordinates to the range from -0.5 to +0.5 and added the matrix transformations.

The fix is included in commit a8ddd61.